707. Design Linked List

PLATFORM	DIFFICULTY	DATE
Platform: LeetCode	MEDIUM	Solved: June 22, 202

Problem Summary

Design a singly linked list from scratch supporting get, addAtHead, addAtTail, addAtIndex, and deleteAtIndex.

Algo

1. get: check bounds, walk index steps, return val.
2. addAtHead: new node's next = head, update head, size++.
3. addAtTail: if empty → addAtHead logic; else walk to last node, attach, size++.
4. addAtIndex: check bounds; if 0 → addAtHead; if size → addAtTail; else walk to index-1, splice in, size++.
5. deleteAtIndex: check bounds; if 0 → move head forward; else walk to index-1, skip target, size--.

Observations

- Same skeleton everywhere: validate → handle head case → walk → relink → update size.
- No dummy node, so index-0 always special-cased.

Section

Time Complexity

- get, addAtIndex, deleteAtIndex: O(index), worst O(n)
- addAtHead: O(1)
- addAtTail: O(n)

Space Complexity

- O(1) per operation.

Key Insights

- size enables O(1) bounds-checking without traversal.
- Index 0 always needs special-casing since there's no prev node.
- Insert/delete both rely on reaching the node just before the target.

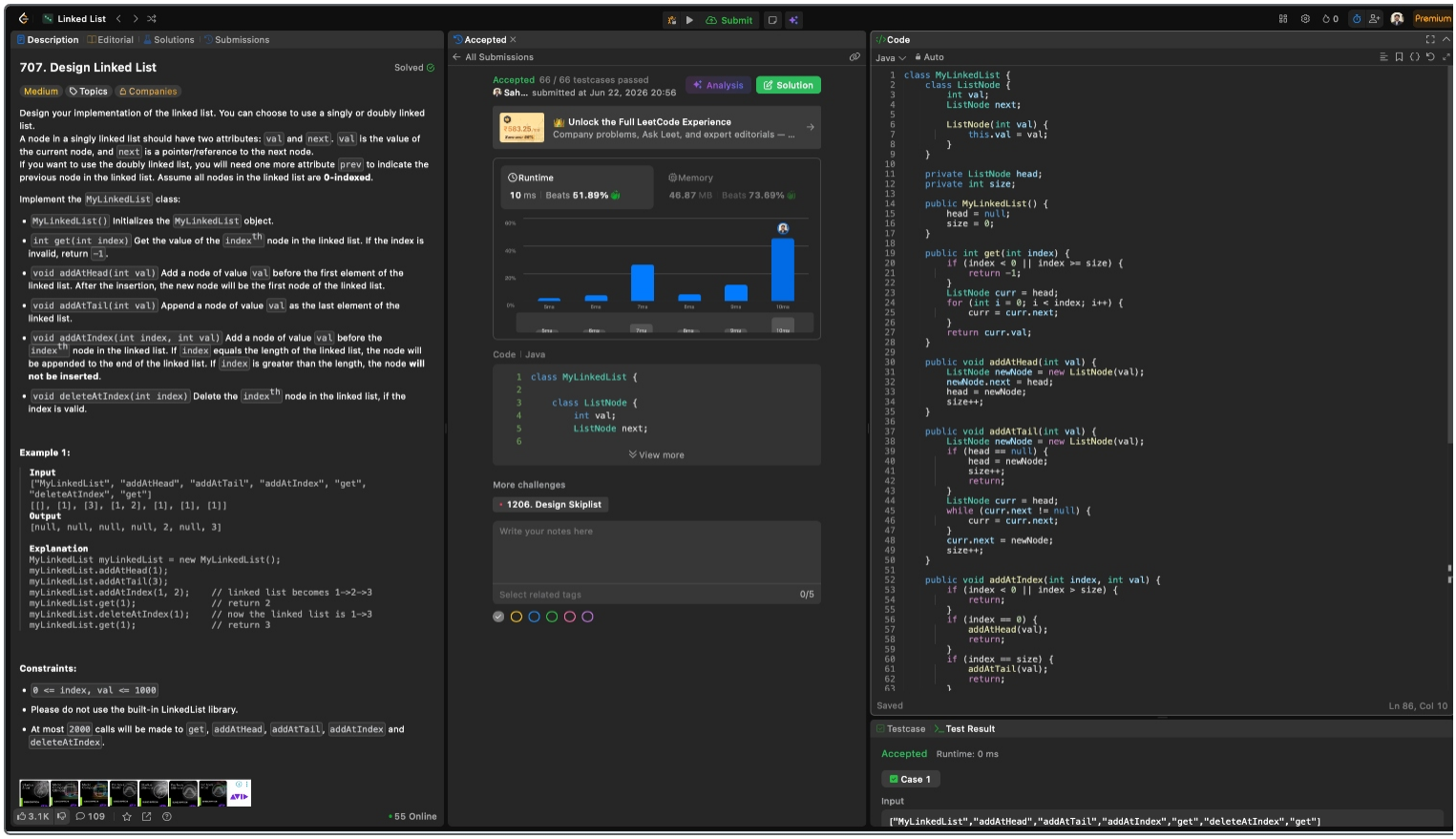
Approach

Maintain a head pointer and a size counter. Each operation validates bounds using size, then either directly modifies head(edge case) or walks via a loop to the required position and relinks pointers.

Code

```
class MyLinkedList {
  class ListNode {
    int val;
    ListNode next;
    ListNode(int val) {
      this.val = val;
    }
  }
  private ListNode head;
  private int size;
  public MyLinkedList() {
    head = null;
    size = 0;
  }
  public int get(int index) {
    if (index < 0 || index >= size) {
      return -1;
    }
    ListNode curr = head;
    for (int i = 0; i < index; i++) {
      curr = curr.next;
    }
    return curr.val;
  }
  public void addAtHead(int val) {
    ListNode newNode = new ListNode(val);
    newNode.next = head;
    head = newNode;
    size++;
  }
  public void addAtTail(int val) {
    ListNode newNode = new ListNode(val);
    if (head == null) {
      head = newNode;
      size++;
      return;
    }
    ListNode curr = head;
    while (curr.next != null) {
      curr = curr.next;
    }
    curr.next = newNode;
    size++;
  }
  public void addAtIndex(int index, int val) {
    if (index < 0 || index > size) {
      return;
    }
    if (index == 0) {
```

```
            addAtHead(val);
            return;
        }
        if (index == size) {
            addAtTail(val);
            return;
        }
        ListNode prev = head;
        for (int i = 0; i < index - 1; i++) {
            prev = prev.next;
        }
        ListNode newNode = new ListNode(val);
        newNode.next = prev.next;
        prev.next = newNode;
        size++;
    }
    public void deleteAtIndex(int index) {
        if (index < 0 || index >= size) {
            return;
        }
        if (index == 0) {
            head = head.next;
            size--;
            return;
        }
        ListNode prev = head;
        for (int i = 0; i < index - 1; i++) {
            prev = prev.next;
        }
        prev.next = prev.next.next;
        size--;
    }
}
```


Linked List
Submit
Premium
Description
Editorial
Solutions
Submissions
707. Design Linked List
Solved
Medium
Topics
Companies
Design your Implementation of the linked list. You can choose to use a singly or doubly linked list.
A node in a singly linked list should have two attributes: val and next. val is the value of the current node, and next is a pointer/reference to the next node.
If you want to use the doubly linked list, you will need one more attribute prev to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.
Implement the MyLinkedList class:
MyLinkedList() Initializes the MyLinkedList object.
int get(int index) Get the value of the indexth node in the linked list. If the index is invalid, return -1.
void addAtHead(int val) Add a node of value val before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
void addAtTail(int val) Append a node of value val as the last element of the linked list.
void addAtIndex(int index, int val) Add a node of value val before the indexth node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted.
void deleteAtIndex(int index) Delete the indexth node in the linked list, if the index is valid.
Example 1:
Input
["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get", "deleteAtIndex", "get"]
[[], [1], [3], [1, 2], [1], [1], [1]]
Output
[null, null, null, null, 2, null, 3]
Explanation
MyLinkedList myLinkedList = new MyLinkedList();
myLinkedList.addAtHead(1);
myLinkedList.addAtTail(3);
myLinkedList.addAtIndex(1, 2); // linked list becomes 1->2->3
myLinkedList.get(1); // return 2
myLinkedList.deleteAtIndex(1); // now the linked list is 1->3
myLinkedList.get(1); // return 3
Constraints:
0 <= index, val <= 1000
Please do not use the built-in LinkedList library.
At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.
3.1K
109
55 Online
Accepted
All Submissions
Accepted 66 / 66 testcases passed
Sah... submitted at Jun 22, 2026 20:56
Analysis
Solution
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — ...
Runtime
10 ms | Beats 51.89%
Memory
46.87 MB | Beats 73.69%
Code | Java
class MyLinkedList {
class ListNode {
int val;
ListNode next;
View more
More challenges
1206. Design Skiplist
Write your notes here
Select related tags
0/5
Code
Java
Auto
class MyLinkedList {
class ListNode {
int val;
ListNode next;
ListNode(int val) {
this.val = val;
}
}
private ListNode head;
private int size;
public MyLinkedList() {
head = null;
size = 0;
}
public int get(int index) {
if (index < 0 || index >= size) {
return -1;
}
ListNode curr = head;
for (int i = 0; i < index; i++) {
curr = curr.next;
}
return curr.val;
}
public void addAtHead(int val) {
ListNode newNode = new ListNode(val);
newNode.next = head;
head = newNode;
size++;
}
public void addAtTail(int val) {
ListNode newNode = new ListNode(val);
if (head == null) {
head = newNode;
size++;
return;
}
ListNode curr = head;
while (curr.next != null) {
curr = curr.next;
}
curr.next = newNode;
size++;
}
public void addAtIndex(int index, int val) {
if (index < 0 || index > size) {
return;
}
if (index == 0) {
addAtHead(val);
return;
}
if (index == size) {
addAtTail(val);
return;
}
Saved
Ln 86, Col 10
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Input
["MyLinkedList","addAtHead","addAtTail","addAtIndex","get","deleteAtIndex","get"]